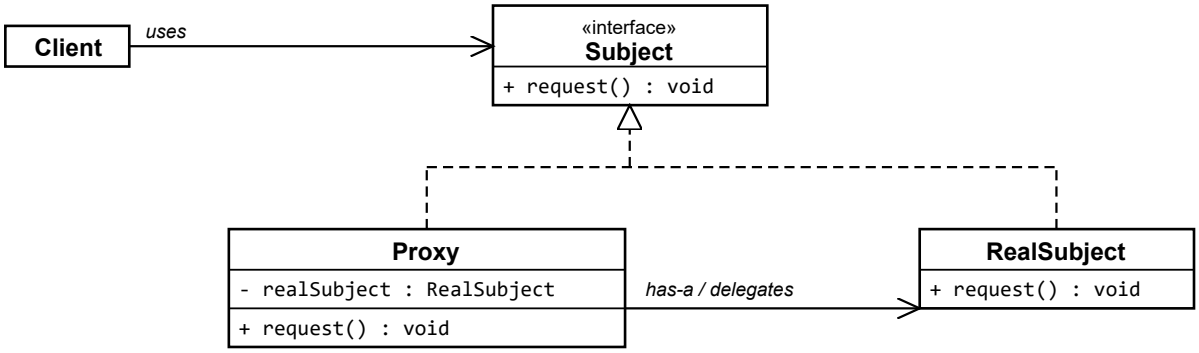
05 Proxy

Structural · “same interface, controlled access”

Intent (GoF)	Provide a **surrogate or placeholder for another object to control access to it**. The proxy can do something **before or after** the request reaches the real object.
Use when	The client must not reach the real object directly: needs **permission checks**, the object is **expensive to create/load** (load only when needed), is **remote**, or calls must be **cached / logged / counted / rate-limited**.
Trigger words	“load only a couple of pages rather than the entire document”, “lazy loading”, “should not directly access; access controlled through verifies permissions”, “restricted archive”, “authorization options / roles”, “cache”, “remote service”, “log every access”.
Participants	**Subject** (common interface) · **RealSubject** (does the real work) · **Proxy** (same interface, holds a reference to RealSubject, controls access, forwards) · **Client** (uses Subject; cannot tell proxy from real).
Types	**Virtual** (lazy loading of heavy objects) · **Protection** (authorization by role) · **Remote** (local representative of a remote object: RMI, gRPC, API gateway) · **Cache/Smart** (cache results, reference counting) · **Logging** (audit) · Security/firewall.

UML — generic



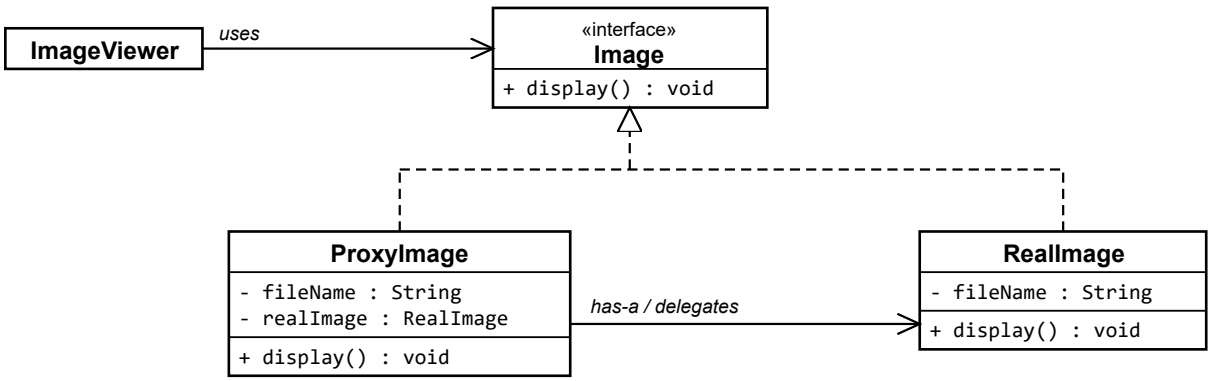
Proxy and RealSubject implement the same interface;
Proxy checks / logs / caches / lazy-loads, then forwards.

Slide quiz statement	True?
Proxy and RealSubject are subtypes of Subject	**Yes** (both implement it)
Polymorphism through implementing the Subject interface	**Yes**
RealSubject has a reference to the Proxy	**No** — Proxy refers to RealSubject
Can’t achieve polymorphism because both implement Subject	**No**

Request flow (sequence)
1. Client calls `request()` on Subject (actually the Proxy)
2. Proxy pre-processing: check access, validate, cache lookup, log
3. If allowed: create RealSubject if needed, forward `request()`
4. RealSubject does the real work, returns
5. Proxy post-processing (cache result, log), returns to client

Why must Proxy and RealSubject share one interface? So the client can use either one without changing code (transparent substitution); the client “should never know which one it receives”.

Virtual proxy (lazy loading) — Image viewer (lecture)



```
public interface Image { void display(); }

public class RealImage implements Image {
    private String fileName;
    public RealImage(String fileName) {
        this.fileName = fileName;
        loadFromDisk();                        // EXPENSIVE
    }
    private void loadFromDisk() { System.out.println("Loading "
+ fileName); }
    public void display() { System.out.println("Displaying " +
fileName); }
}
```

```
public class ProxyImage implements Image {
    private String fileName;
    private RealImage realImage;              // null until
needed
    public ProxyImage(String fileName) { this.fileName =
fileName; }
    public void display() {
        if (realImage == null)
            realImage = new RealImage(fileName);   // load on
first use
        realImage.display();                       // forward
    }
}
Image img = new ProxyImage("photo.jpg");   // nothing loaded yet
img.display();   // Loading photo.jpg, Displaying photo.jpg
img.display();   // Displaying photo.jpg (already loaded)
```

Protection proxy — Bank withdrawal / hospital roles (lecture)

```
public interface Account { void withdraw(double amount); }

public class BankAccount implements Account {          // real
    private double balance = 50000;
    public void withdraw(double amt) { balance -= amt;
        System.out.println("Withdrawn " + amt); }
}
public class AccountProxy implements Account {         // proxy
    private BankAccount real = new BankAccount();
    private String pin; private boolean active;
    private double dailyLimit = 20000;
    public AccountProxy(String pin, boolean active) { this.pin =
pin; this.active = active; }
    public void withdraw(double amt) {
        if (!"1234".equals(pin))   { System.out.println("Invalid
PIN"); return; }
        if (!active)               { System.out.println("Account
blocked"); return; }
        if (amt > dailyLimit)      { System.out.println("Over
limit"); return; }
        System.out.println("[LOG] withdraw " + amt);  // logging
        real.withdraw(amt);                            //
forward
    }
}
```

Role	`viewPatient()`
Doctor	allowed → forwarded to PatientService
Receptionist	denied by proxy
Administrator	allowed

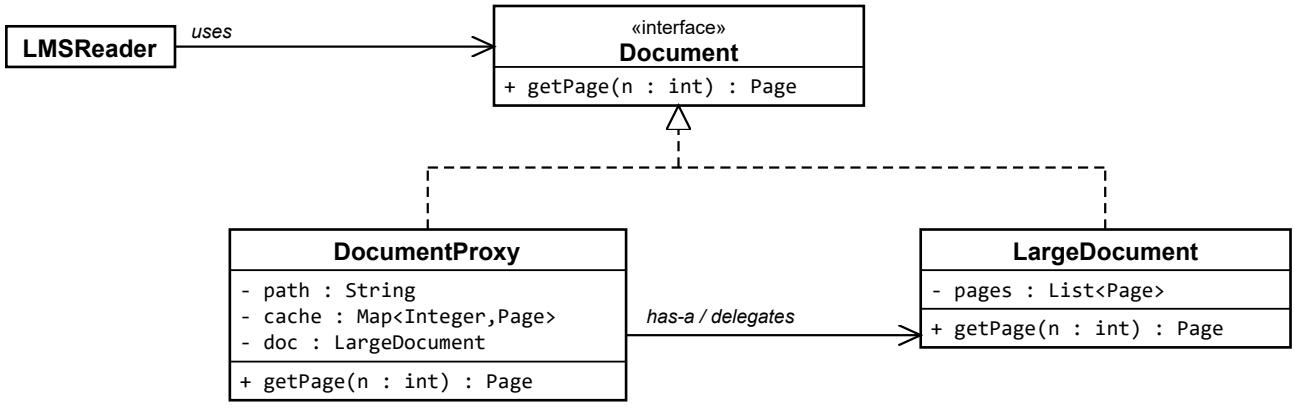
All users call the same method; the **proxy decides** using the role-access matrix. The real `PatientService` only handles allowed requests.
Remote proxy: mobile banking app calls `getBalance()` on a local proxy that sends HTTP/JSON to the bank server and returns the result — the app "feels like calling a local object". **API Gateway** = proxy in microservices (auth, rate limiting, routing, caching).
Warehouse example: proxy checks stock and routes orders only to warehouses that can fulfil them.

Advantages / disadvantages / challenge → mechanism

Advantages	Disadvantages	Challenge → mechanism
Security: controlled access to the real object	Extra layer ⇒ response delay	Overhead for cheap/public objects → don't use a proxy there (calculator, static menu)
Performance: lazy loading, caching	More classes, more complexity	Stale cache → expiry time / invalidate on update
Client code unchanged (same interface, OCP)	Proxy and real must be kept in sync when interface changes	Thread-safe lazy creation → synchronize the creation check
Logging/auditing, remote access transparency	Debugging through layers is harder	Proxy becomes a bottleneck / single point of failure → replicate proxies (HA), keep them stateless

Questions & model answers

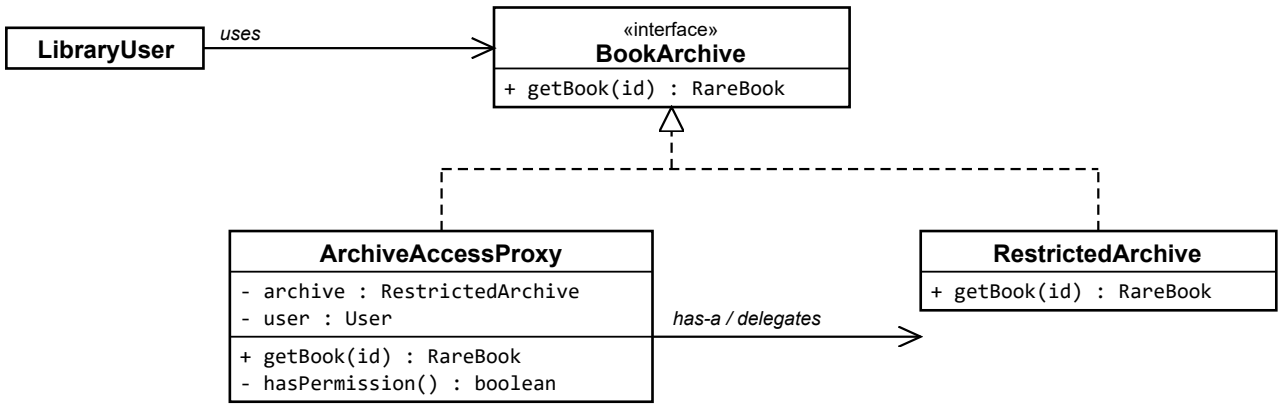
LMS: when users read large content, load only a couple of pages rather than the entire document. *2022 Q4.4, 2023 Q4.4 (5)*



Virtual Proxy. `DocumentProxy` implements `Document` like `LargeDocument`; the reader holds the proxy. `getPage(n)` loads only the requested page (and maybe the next one) from storage and caches it; the full document is never loaded unless needed ⇒ fast opening, low memory/bandwidth. The reader code doesn't change.

```
public class DocumentProxy implements Document {
    private final String path; private final Map<Integer, Page> cache = new HashMap<>();
    public DocumentProxy(String path) { this.path = path; }
    public Page getPage(int n) {
        return cache.computeIfAbsent(n, k -> PageLoader.load(path, k));   // load one page on demand
    }
}
```

Rare books in a restricted archive; users must not access them directly; access controlled through permission checks. Explain and give an example. *2025 Q3.1(c), 3.5 (5)*



Protection Proxy. `ArchiveAccessProxy` and `RestrictedArchive` implement `BookArchive`. The proxy's `getBook(id)` first checks the user's role/permission (e.g. researcher with approved request, membership valid), logs the access, then forwards to the real archive; otherwise throws `AccessDeniedException`. Example: a student requests a 1820 manuscript → denied; a librarian-approved researcher → allowed and logged.

```
public RareBook getBook(String id) {
    if (!user.hasRole("RESEARCHER") || !user.isApproved(id)) throw new AccessDeniedException("No permission");
    AuditLog.record(user, id);            // logging proxy duty
    return archive.getBook(id);           // forward to real subject
}
```

Website with different authorization options depending on the user's need. *2023 Q4.5 (5)*
Protection Proxy in front of each protected service (`AdminServiceProxy`, `ReportServiceProxy`) checks role/permissions before forwarding. If authorization is a *sequence* of checks (login → role → permission → 2FA), combine with Chain of Responsibility. If features are optional add-ons per user, Decorator.

Implementing a logging mechanism — which pattern? *2023 Q3.4 (5)*
Two valid views: **Singleton** for the logger itself (one shared logger writing to one file — see Note 01). To **add logging around existing services without changing them**, a **Proxy** (logging proxy) intercepts each call, logs before/after, and forwards. State which you mean and why.

Proxy vs Decorator vs Adapter
Proxy = **same interface, controls access** (usually one layer, may create the real object itself). Decorator = same interface, **adds responsibilities**, stacked many times, wrapped object given by client. Adapter = **different interface**, converts. "A protection proxy might be implemented like a decorator, but remember the intent."

Use proxy? Hospital login (yes, protection) · image editor high-res (yes, virtual) · online exam authentication (yes, protection) · CDN cache (yes, cache) · Netflix streaming (protection + cache) · Google Drive thumbnails (virtual + remote) · calculator / static restaurant menu (**no** — no access control or expensive load). **Mistakes:** proxy with a different interface (that's Adapter) · arrow from RealSubject to Proxy · client creating RealSubject directly.